Trap Handling in RV32 S-mode

Control and status registers, cause decoding, and a verified trap_dispatch() for rivOS

1. The key idea

Never identify a trap from stval alone. Decode **scause** first. It tells you *why* the trap happened. Then use **sepc** (where) and **stval** (extra detail whose meaning depends on the cause).

CSR	Question it answers	Used in trap_dispatch for
scause	Why did the trap happen?	Selecting the handler (interrupt vs exception, then the code)
sepc	At which instruction?	Return address; add 4 after a handled ecall
stval	What extra detail is there?	Faulting address, or illegal instruction bits (cause-dependent)
sstatus	What were the privilege and interrupt states?	Saved/restored by the trap frame; SPP says U or S origin
sie / sip	Which interrupts are enabled / pending?	Enabling the timer (STIE); diagnosing pending IRQs

2. Verification summary

Every claim in the original notes was checked against the RISC-V Privileged Architecture, Supervisor-Level ISA chapter (v1.13), and the C code was compiled with `gcc -Wall -Wextra`. **Two factual errors** were found in the sstatus bit table (FS and XS positions). Everything else was correct, with a few additions and clarifications.

#	Claim in original notes	Status	Result
1	scause bit 31 = interrupt flag on RV32; low bits = code	Verified	Matches spec (Interrupt bit + Exception Code field).
2	Interrupt codes 1 (SSI), 5 (STI), 9 (SEI)	Verified	Matches spec. Code 13 (counter overflow, Sscofpmf) also exists.
3	Exception codes 0-9, 12, 13, 15; 10-11 and 14 reserved	Verified	Matches spec Table 2 exactly.
4	"16+ extension-dependent"	Clarified	Spec: 16-17 reserved, 18 = software check, 19 = hardware error, 20-23 reserved, 24-31 and 48-63 custom.
5	stval meaning per cause	Verified	Spec: nonzero stval on breakpoint, misaligned, access fault, page fault = faulting virtual address. Illegal instruction may hold instruction bits. All other traps set it to zero.
6	stval == 0 does not mean "no exception"	Verified	Spec allows zero, and hardware may choose to always write zero.
7	sstatus: SIE=bit 1, SPIE=bit 5, SPP=bit 8, SD=bit 31	Verified	Matches RV32 layout.
8	sstatus: FS = bits 13:12	Corrected	FS is bits **14:13**.
9	sstatus: XS = bits 18:17	Corrected	XS is bits **16:15**. (Bits 18 and 19 are SUM and MXR.)
10	sie/sip: SSIE/SSIP=1, STIE/STIP=5, SEIE/SEIP=9	Verified	Matches spec. Added: STIP and SEIP are read-only in sip; SSIP is writable.
11	stvec Direct/Vectored behavior	Verified	Matches spec. Vectored timer IRQ lands at BASE + 0x14. BASE must be 4-byte aligned.
12	On trap: SPIE=SIE, SIE=0, SPP set; sret reverses	Verified	Matches spec (sret: SIE=SPIE, SPIE=1, SPP=U).
13	ecall: set sepc = sepc + 4 after handling	Verified	sepc holds the address of the ecall itself, and ecall is 4 bytes.
14	scountinhibit listed as supervisor CSR	Clarified	Not in the base supervisor chapter. It comes from optional counter-delegation extensions. Ignore it for now.
15	S-mode ecall (cause 9) -> panic	Clarified	Normally S-mode ecall goes to M-mode/SBI, so cause 9 rarely reaches your stvec handler. Panic is a safe default.
16	Frame offsets 124/128/132/136 for indices 31-34	Verified	Arithmetic correct (index x 4). The frame layout itself depends on your trap.s, which I cannot see.
17	trap_dispatch C skeleton	Verified	Compiles with no warnings (gcc -std=c11 -Wall -Wextra). Hardened version in section 8.

Legend: green = verified, red = corrected, amber = clarified, blue = added.

3. scause: decoding the trap

On RV32, scause is 32 bits wide. Bit 31 is the Interrupt flag; bits 30:0 hold the Exception Code.

```
uint32_t is_interrupt = scause >> 31;
uint32_t code         = scause & 0x7fffffffU;
```

3.1 Interrupts (scause bit 31 = 1)

Code	Name	Typical OS action
1	Supervisor software interrupt (SSI)	Inter-processor notification, if used
5	Supervisor timer interrupt (STI)	Re-arm the timer (SBI TIME), then consider scheduling. This is your scheduler tick.
9	Supervisor external interrupt (SEI)	Ask the interrupt controller which device fired; run its handler
13	Counter-overflow interrupt (needs Sscofpmf)	Performance-counter overflow; ignore unless used
others	Reserved, or platform use (16 and above)	Treat as unexpected

When several supervisor interrupts are pending, the priority order is SEI, then SSI, then STI, then counter-overflow.

3.2 Exceptions (scause bit 31 = 0)

Code	Exception	Cause	stval holds
0	Instruction address misaligned	Jump or branch target breaks alignment	Faulting address (or 0)
1	Instruction access fault	Fetch denied by PMP/PMA or failed	Faulting address (or 0)
2	Illegal instruction	Unknown, privileged, or disabled instruction/CSR access	Instruction bits (or 0)
3	Breakpoint	ebreak / c.ebreak	Address (or 0)
4	Load address misaligned	Misaligned load	Faulting address (or 0)
5	Load access fault	Load blocked by PMP/PMA or bus error	Faulting address (or 0)
6	Store/AMO address misaligned	Misaligned store or atomic	Faulting address (or 0)
7	Store/AMO access fault	Store or atomic blocked or bus error	Faulting address (or 0)
8	Environment call from U-mode	User program executed ecall (system call)	0
9	Environment call from S-mode	Kernel executed ecall (usually goes to SBI instead)	0
10-11	Reserved	Not defined in the S-level table	-
12	Instruction page fault	Fetch failed Sv32 translation or permission check	Faulting virtual address
13	Load page fault	Load failed translation or permission check	Faulting virtual address
14	Reserved	Not defined	-
15	Store/AMO page fault	Store/atomic failed translation or permission check	Faulting virtual address
16-17	Reserved	-	-
18	Software check	Newer extensions (e.g. control-flow integrity)	Extension-defined
19	Hardware error	Platform-reported hardware error	Platform-defined
20-23	Reserved	-	-
24-31, 48-63	Custom use	Vendor-specific	Vendor-defined

"(or 0)" means the platform is allowed to write zero instead. Never rely on stval being nonzero for diagnosis alone.

4. stval: what the value means

There is **no universal stval value** that identifies an exception. The meaning depends on scause. The spec's rules:

Trap type	stval contents	Example
Breakpoint, misaligned, access fault, page fault	Faulting **virtual address** if nonzero. For a misaligned access that then faults, the address of the portion that faulted.	Load page fault at 0x40002000: scause=13, stval=0x40002000
Instruction fetch fault on variable-length code	Address of the **portion** of the instruction that faulted. sepc points to the start of the instruction.	Compressed or straddling instruction
Illegal instruction (cause 2)	Optionally the faulting **instruction bits** (right-justified, upper bits zero), else 0.	stval=0x00000073 if the bits were reported
ecall (8, 9) and all interrupts	**Zero**.	Use a0-a7 and sepc, not stval

Example: a load page fault on virtual address 0x40002000:

Register	Value	Meaning
scause	0x0000000D	Exception, code 13 = load page fault
sepc	address of the load instruction	Where the faulting instruction is
stval	0x40002000	The virtual address that could not be translated

5. Supervisor CSRs you need

5.1 Trap setup and handling

CSR	Purpose	How rivOS uses it
sstatus	Supervisor status	Interrupt-enable and previous-privilege state, restored by sret
sie	Interrupt enables	Set STIE (bit 5) to allow the timer interrupt
stvec	Trap entry address and mode	Point to trap_entry (Direct mode first)
sscratch	Scratch register for the kernel	Swap with sp at trap entry to reach the kernel stack
sepc	Trap return PC	Saved in the frame; restored by sret
scause	Trap cause	Selects the handler in trap_dispatch
stval	Extra trap value	Diagnostics, page-fault address
sip	Interrupt pending	STIP is read-only; set by the SBI/hardware

5.2 Memory and optional CSRs

CSR	Purpose	When you need it
satp	MODE, ASID, root page-table PPN	Sv32 virtual memory. Bit 31 = MODE (0 Bare, 1 Sv32), bits 30:22 = ASID, bits 21:0 = PPN. After changing page tables, run sfence.vma.
scounteren	Which counters U-mode may read	Only if user code reads cycle/time/instret. A clear bit makes the read an illegal-instruction trap.
senvcfg	User execution-environment controls	Only for extensions that use it (e.g. fence behavior)
stimecmp	Timer compare (Sstc extension)	Only if Sstc exists. RV32 also needs stimecmph. Not needed when using SBI TIME.

6. Bit layouts

6.1 sstatus on RV32 (corrected)

Bit(s)	Name	Meaning
1	SIE	Global supervisor interrupt enable. Cleared on trap entry.
5	SPIE	Value of SIE before the trap
6	UBE	U-mode data endianness (0 = little-endian)
8	SPP	Privilege before the trap: 0 = U-mode, 1 = S-mode
10:9	VS	Vector extension state (if implemented)
14:13	FS	Floating-point state (Off/Initial/Clean/Dirty). *Original notes said 13:12.*
16:15	XS	Other-extension state. *Original notes said 18:17.*
18	SUM	S-mode may access U-mode pages
19	MXR	Loads may read execute-only pages
31	SD	Summary: FS, VS or XS is dirty

6.2 What the hardware does at trap entry and at sret

Event	Hardware actions
Trap into S-mode	sepc = address of trapping instruction; scause and stval written; SPIE = SIE; SIE = 0; SPP = previous privilege (0 if from U, 1 if from S); pc = stvec target
sret	pc = sepc; privilege = SPP (U if 0, S if 1); SIE = SPIE; SPIE = 1; SPP = 0

Do not set SPP by hand to choose a process. Save and restore the full sstatus value for the context you resume.

6.3 sie and sip bits

Bit	sie name	sip name	Notes
1	SSIE	SSIP	SSIP is writable by S-mode (software interrupt)
5	STIE	STIP	STIP is read-only; the SBI/hardware sets and clears it
9	SEIE	SEIP	SEIP is read-only; set by the interrupt controller

An interrupt traps to S-mode only if (a) you are in S-mode with sstatus.SIE = 1, or you are in U-mode, **and** (b) its bit is set in both sip and sie. Enabling STIE alone does not enable interrupts; the timer must also be programmed. Example: `li t0,0x20 ; csrs sie,t0` sets STIE.

6.4 stvec modes

MODE (bits 1:0)	Name	Behavior
0	Direct	Every trap sets pc = BASE
1	Vectored	Exceptions: pc = BASE. Interrupts: pc = BASE + 4 x cause (timer = BASE + 0x14)
2-3	Reserved	Do not use

Use Direct mode first: all traps enter at one address, which is easier to debug.

7. Trap frame layout

The dispatcher reads the frame that trap.s saved. The C indices and the assembly offsets must agree exactly.

Field	Word index	Byte offset	Status
x1 ... x31 (saved registers)	0 ... 30	0 ... 120	Assumed: x0 is not saved
sepc	31	124	Offset arithmetic verified
sstatus	32	128	Offset arithmetic verified
scause	33	132	Offset arithmetic verified
stval	34	136	Offset arithmetic verified

The frame is 35 words = 140 bytes. Allocate **144 bytes** (or any multiple of 16) so sp stays 16-byte aligned, as the RISC-V ABI requires. The register part of the layout is an assumption about your trap.s, which I could not see.

8. trap_dispatch() decision flow

Step	Check	Action
1	Read epc, cause, tval from the frame	Frame indices 31, 33, 34
2	cause bit 31 = 1 (interrupt)?	Go to step 3; otherwise step 4
3	Interrupt code	5: timer_rearm() then scheduler_request(). 1 and 9: handle only if used. Anything else: panic. Return the frame.
4	Exception code 8 (U ecall)	Run the system call, set frame sepc = epc + 4, return the frame
5	All other exceptions	panic_trap(cause, epc, tval). Page-fault recovery can be added later.

8.1 Verified code (kernel/trap.c)

Compiled with `gcc -std=c11 -Wall -Wextra -fsyntax-only`: no errors or warnings. Changes from the original: named frame indices, `noreturn` on panic_trap, and the duplicate panic cases merged into one fall-through.

```
#include <stdint.h>

#define SCAUSE_INTERRUPT  0x80000000U
#define SCAUSE_CODE_MASK  0x7fffffffU

#define CAUSE_INST_MISALIGNED  0
#define CAUSE_INST_ACCESS      1
#define CAUSE_ILLEGAL_INST     2
#define CAUSE_BREAKPOINT       3
#define CAUSE_LOAD_MISALIGNED  4
#define CAUSE_LOAD_ACCESS      5
#define CAUSE_STORE_MISALIGNED 6
#define CAUSE_STORE_ACCESS     7
#define CAUSE_U_ECALL          8
#define CAUSE_S_ECALL          9
#define CAUSE_INST_PAGE_FAULT  12
#define CAUSE_LOAD_PAGE_FAULT  13
#define CAUSE_STORE_PAGE_FAULT 15

#define IRQ_SUPERVISOR_SOFT    1
#define IRQ_SUPERVISOR_TIMER   5
#define IRQ_SUPERVISOR_EXT     9

/* Frame word indices (must match kernel/trap.s): x1..x31 = 0..30 */
#define FR_SEPC     31
#define FR_SSTATUS  32
#define FR_SCAUSE   33
#define FR_STVAL    34

extern void timer_rearm(void);
extern void scheduler_request(void);
extern void panic_trap(uint32_t cause, uint32_t epc, uint32_t tval)
    __attribute__((noreturn));

uint32_t *trap_dispatch(uint32_t *frame)
{
    uint32_t epc   = frame[FR_SEPC];
    uint32_t cause = frame[FR_SCAUSE];
    uint32_t tval  = frame[FR_STVAL];

    uint32_t is_interrupt = (cause & SCAUSE_INTERRUPT) != 0;
    uint32_t code = cause & SCAUSE_CODE_MASK;

    if (is_interrupt) {
        switch (code) {
        case IRQ_SUPERVISOR_TIMER:
            timer_rearm();          /* program next deadline first */
            scheduler_request();    /* later: may pick another frame */
            break;
        case IRQ_SUPERVISOR_SOFT:   /* only if software IRQs are used */
            break;
        case IRQ_SUPERVISOR_EXT:    /* only if external IRQs are used */
            break;
        default:
            panic_trap(cause, epc, tval);
        }
        return frame;
    }

    switch (code) {
    case CAUSE_U_ECALL:
        /* ...system-call dispatch here... then skip the 4-byte ecall */
        frame[FR_SEPC] = epc + 4;
        break;
    case CAUSE_S_ECALL:
    case CAUSE_BREAKPOINT:
    case CAUSE_INST_PAGE_FAULT:
    case CAUSE_LOAD_PAGE_FAULT:
    case CAUSE_STORE_PAGE_FAULT:
    case CAUSE_INST_MISALIGNED:
    case CAUSE_INST_ACCESS:
    case CAUSE_ILLEGAL_INST:
    case CAUSE_LOAD_MISALIGNED:
    case CAUSE_LOAD_ACCESS:
    case CAUSE_STORE_MISALIGNED:
    case CAUSE_STORE_ACCESS:
    default:
        panic_trap(cause, epc, tval);   /* does not return */
    }
    return frame;
}
```

9. Issues to resolve before context switching

#	Issue	What to do
1	Frame layout mismatch between assembly and C	Define the layout once (shared header or macros) and add a compile-time size check
2	A timer interrupt must not blindly switch stacks	scheduler_request() should pick a saved frame; trap_dispatch() returns that pointer; assembly restores that frame, including its sepc and sstatus
3	Traps from U-mode need a kernel stack	Swap sp with sscratch at entry, use SPP to know the origin, and switch to a kernel stack before saving the frame
4	Timer must be programmed before STIE is enabled	Call SBI set_timer first, then set sie.STIE, then sstatus.SIE
5	panic_trap must not return	Print scause, sepc, stval, then halt

Recommended order

Step	Goal	Done when
1	Install stvec (Direct) and trigger a deliberate illegal instruction	panic_trap prints scause=2 and the right sepc
2	Enable the timer and return the same frame	Ticks repeat forever without a crash
3	Add U-mode ecall handling	Return from ecall continues at the next instruction
4	Connect scheduler_request() to the process table	Different frames are returned and processes alternate

Sources

RISC-V Instruction Set Manual, Volume II: Privileged Architecture, chapter "Supervisor-Level ISA" (version 1.13), docs.riscv.org. Used for the scause table, stval rules, stvec, sip/sie, sepc, satp and the SPIE/SIE/SPP behavior. FS/XS/SD bit positions were cross-checked against published RV32 mstatus layouts (the sstatus register is a subset of mstatus).